



华中科技大学

MMX、SSE、AVX 指令集

许向阳

xuxy@hust.edu.cn





指令系统的发展

SIMD: Single Instruction Multiple Data

MMX : Multi Media eXtension 1997 Pentium II

SSE : Streaming SIMD Extension 1999

SSE2 (2001) SSE3 (2004) SSE4 (2008)

AVX : Advanced Vector eXtensions

- 2011年, Intel酷睿处理器i3、i5、i7系列中使用
- 2013年, Intel发布了 AVX2
- 2016年推出了AVX-512
酷睿i7、i9等CPU中都支持AVX-512





指令系统的发展

目前已进入 AVX10+APX+AMX 的 “三驾马车” 时代。

➤ AVX10 统一向量架构

➤ APX 革新通用性能

从 16 个通用寄存器 (GPR) 增至 32 个

➤ AMX 深耕 AI 加速

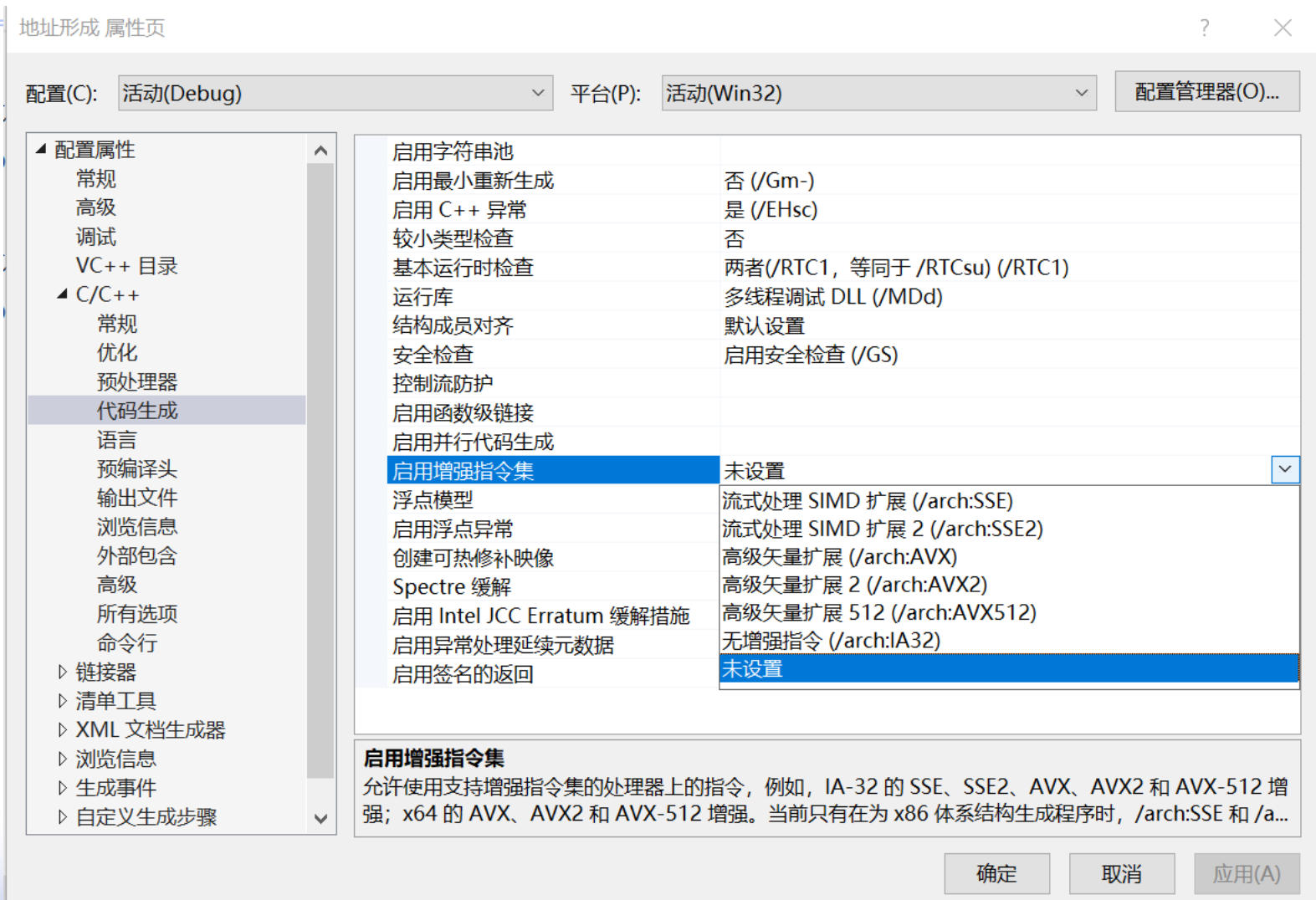
AMX 是独立于 AVX 的矩阵加速单元，专为深度学习设计，性能远超 AVX-512

三者协同发力，覆盖从消费级到数据中心的全场景。





VS 中使用不同指令系统的开关





MMX程序设计

1 MMX技术简介

单指令多数据流的基本概念

MMX寄存器

环绕运算、饱和运算

2 MMX指令简介

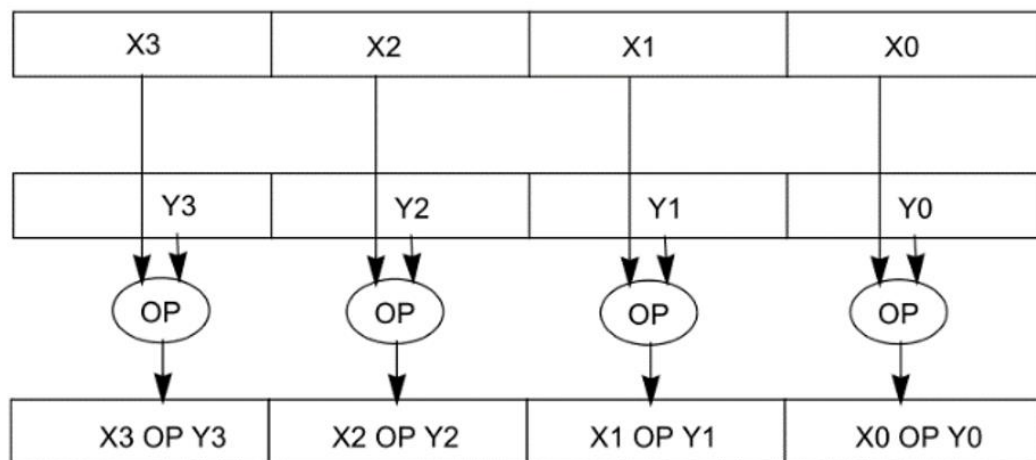
3 MMX编程示例

4 用C语言编写MMX应用程序



1 MMX技术简介

➤ 单指令多数据流 SIMD



	10 (000AH)	<u>20</u> (0014H)	<u>30</u> (001EH)	40 (0028H)
+	25 (0019H)	-1 (0FFFFH)	-35 (0FFDDH)	35 (0023H)
	35 (0023H)	<u>19</u> (0013H)	-5 (0FFFBH)	75 (004BH)



1 MMX技术简介

环绕运算

```
MOV  AL, 7FH
ADD  AL, 1      (AL)=80H
MOV  AL, 0FFH
ADD  AL, 1      (AL)=0
```

一个水杯装满了水，再向其中加水，结果如何？
一个空杯，要求从中减（取出）水，结果如何？

饱和运算

为什么会有饱和运算的需求？（图像处理等领域的应用）
饱和运算又分为有符号和无符号的饱和运算





1 MMX技术简介

MMX寄存器

- 8个64位的寄存器：mm0～mm7
- 直接使用这些寄存器：**寄存器寻址**方式。
- **不能用于寄存器间接寻址**、变址寻址和基址加变址寻址，即不能用于寻址内存中的操作数
- 可以用于整数运算，又可以用于浮点运算
- 可存放8个字节、或者4个字、或者2个双字数据
- 是浮点寄存器 ST0-ST7 （80位）的部分





1 MMX技术简介

- 8个64位的寄存器: mm0~mm7



寄存器

EDI = 0019FEE4 EIP = 00415599 ESP = 0019FDE0 EBP = 0019FEE4 EFL = 00000212

CS = 0023 DS = 002B ES = 002B SS = 002B FS = 0053 GS = 002B

MM0 = 0000000000000000 MM1 = 0000000000000000 MM2 = 0000000000000000

MM3 = 0000000000000000 MM4 = 0000000000000000 MM5 = 0000000000000000

MM6 = 0000000000000000 MM7 = 0000000000000000





2 MMX指令简介

数据传送

算术运算

比较运算

逻辑运算

移位、转换、解组

状态控制

常用字母后缀来标识需要处理的元素大小，
b、w、d、q分别对应字节、字、双字、四字。





2 MMX指令简介

指令类别	指令助记符	功能
加法运算(环绕)	paddb , paddw , paddd	字节、字、双字的环绕加法
加法运算(有符号饱和)	paddsb, paddsw	有符号打包整数饱和加法
加法运算(无符号饱和)	paddusb, paddusw	无符号打包整数饱和加法
减法运算(环绕)	psubb , psubw , psubd	字节、字、双字的环绕减法
减法运算(有符号饱和)	psubsb, psubsw	有符号打包整数饱和减法
减法运算(无符号饱和)	psubusb, psubusw	无符号打包整数饱和减法
乘法	pmullw, pmulhw	有符号打包整型的字乘法, 每一对单字相乘的结果为双字, L/H是保留双字的低/高字
乘后加法	pmaddwd	有符号打包整型字乘法, 产生4个双字乘积, 相邻双字相加得2个双字





2 MMX指令简介

比较运算

① pcmpeqb/ pcmpeqw/ pcmpeqd

字节/字/双字 比较 各个对应元素是否相等

若相等，目的操作数中的对应元素被置为全1（即FF/FFFF/FFFFFFFF），否则被置为全0。

② pcmpgtb/ pcmpgtw/ pcmpgtd

有符号打包整型数中的各对应元素比较

若目的操作数大于对应的源操作数，则目的操作数被置为全1，否则被置为全0。





2 MMX指令简介

- 只支持整数运算
- 浮点运算时要进行状态切换



2 MMX指令简介

```
x      db    70H, 0A0H, 50H,  50H, 0F0H, 0F0H, 0F0H, 0F0H
y      db    0A0H, 70H, 30H, 0F0H, 01H, 20H, 81H, 0F0H
movq   mm0, qword ptr x ; mm0 = F0 F0 F0 F0 50 50 A0 70
movq   mm1, qword ptr y ; mm1 = F0 81 20 01 F0 30 70 A0
paddb  mm0, mm1          ; mm0 =  E0 71 10 F1 40 80 10 10
paddsb mm0, mm1          ; mm0 =  E0 80 10 F1 40 7F 10 10
paddusb mm0, mm1         ; mm0 =  FF FF FF F1 FF 80 FF FF
```

paddb: 打包字节整数环绕加法指令

ADD Packed Byte integers

paddsb: 有符号饱和字节加法指令

ADD Packed Signed Byte integers with signed saturation

paddusb: 无符号饱和字节加法指令

ADD Packed UnSigned Byte integers with unsigned saturation





3 MMX编程示例

实现两个向量的内积

设有向量 $a=(a_1, a_2, a_3, a_4)$ ，向量 $b=(b_1, b_2, b_3, b_4)$ 。

向量 a 、 b 的内积为 $\langle a, b \rangle = a_1*b_1 + a_2*b_2 + a_3*b_3 + a_4*b_4$ 。

.686P

.MMX

.model flat, c

ExitProcess proto stdcall :DWORD

printf proto :vararg

includelib libcmt.lib

includelib legacy_stdio_definitions.lib

.data

buf1 sword 1, -2, 3, 400H

buf2 sword 2, 3, 4, 500H

buf3 sdword 0, 0

lpFmt db "%d %x(H)", 0dh, 0ah, 0

.stack 200





3 MMX编程示例

```
.code
main proc
    movq    mm0,    qword ptr    buf1        ; mm0=040000003FFFE0001H
    movq    mm1,    qword ptr    buf2        ; mm1=05000000400030002H
    pmaddwd mm0,    mm1                      ; mm0=0014000CFFFFFFFFCH
    movq    qword ptr buf3, mm0
    mov     eax,    buf3
    add     eax,    buf3+4
    emms
    invoke  printf, offset lpFmt, eax, eax
    invoke  Exitprocess, 0
```

```
main endp
end
```

```
1, -2, 3, 400H
2, 3, 4, 500H
2, -6, 0CH, 140000H
-4 => FFFFFFFC 0014000C
```





4 用C语言编写MMX应用程序

```
#include <stdio.h>      #include <time.h>
#include <stdlib.h>      #include <conio.h>
#define LEN 100000 // 数组大小
int main() {
    clock_t stTime, edTime;
    int i, j;
    unsigned short a[LEN], b[LEN], c[LEN];
    srand(time(NULL));
    for (i = 0; i < LEN; i++) // 生成随机数组
    { a[i] = rand() ; b[i] = rand(); }
    stTime = clock();
    for (j = 0; j < 1000; j++) { // 重复做1000遍
        for (i = 0; i < LEN; i++)
            c[i] = a[i] + b[i];
    }
    edTime = clock(); // edTime - stTime;
    .....
}
```





4 用C语言编写MMX应用程序

```
#include <mmintrin.h>
__m64 *pa, *pb, *pc; // 指向数组 a
int LEN4; // 一次运算4个数，总循环次数减少
for (j = 0; j < 1000; j++) {
    pa = (__m64 *)a;
    pb = (__m64 *)b;
    pc = (__m64 *)c;
    LEN4 = LEN / 4;
    for (i = 0; i < LEN4; i++) {
        *pc = _m_paddw(*pa, *pb);
        pa += 1; // 反汇编后，地址是加 8
        pb += 1;
        pc += 1;
    }
}
_m_empty(); // 实际是 EMMS指令
```





MMX 总结

单指令多数据流的基本概念

环绕运算

有符号/无符号的饱和运算

采用 MMX 指令提高程序运行速度





華中科技大學

SSE指令集





SSE 指令集

- 1 SSE技术简介
- 2 SSE指令简介
- 3 SSE2及后续版本的指令简介
- 4 SSE编程示例
- 5 用C语言编写SSE应用程序





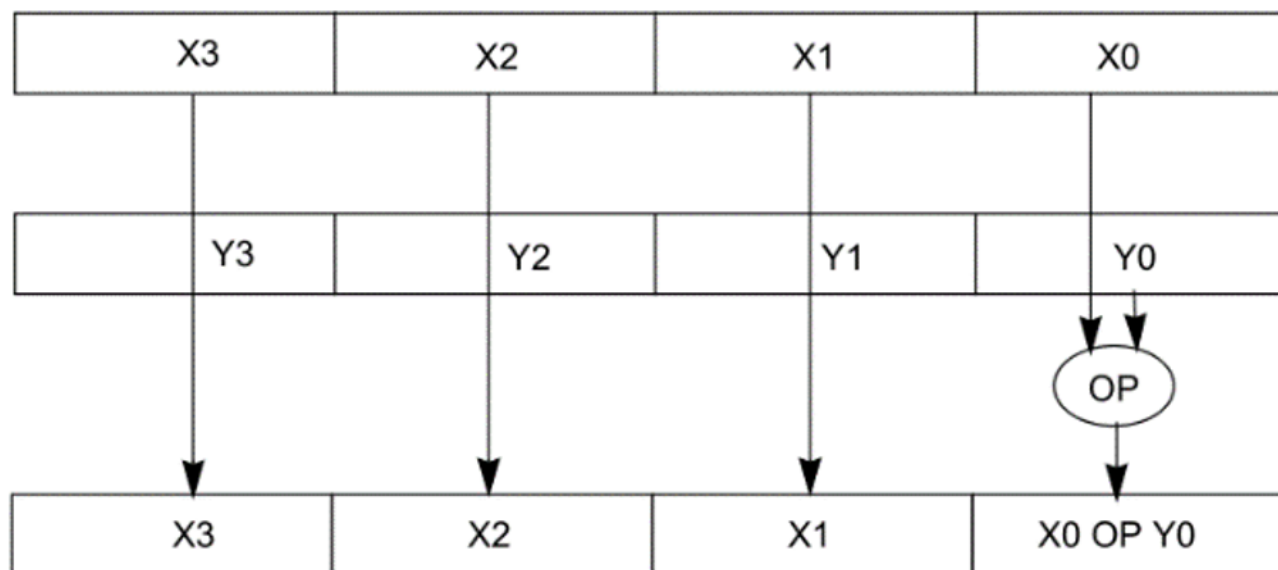
1 SSE技术简介

- Pentium III 中，引入了流式SIMD扩展 SSE
Streaming SIMD Extensions
- 8个128位的寄存器：xmm0~xmm7
- 保留了MMX的64位寄存器对于组合整数进行运算
- 增加了单精度浮点数打包运算的指令
- 增加了标量单精度浮点数运算指令



1 SSE技术简介

➤ 标量单精度浮点数运算





1 SSE技术简介

SSE中的数据寄存器

- 8个128位的寄存器: $xmm0 \sim xmm7$
- 直接使用这些寄存器: 寄存器寻址方式。
- 不能用于寄存器间接寻址、变址寻址和基址加变址寻址, 即不能用于寻址内存中的操作数
- 可以用于整数运算, 又可以用于浮点运算





1 SSE技术简介

SSE中的控制和状态寄存器

31	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留 (Reserved)		FTZ	RC		PM	UM	OM	ZM	DM	IM	DAZ	PE	UE	OE	ZE	DE	IE



2 SSE指令简介

SSE指令可以分为四类：

- 组合和标量单精度浮点指令
- 64位SIMD整数指令
- 状态管理指令
- 其他指令（Cache控制、预取、内存排序）



2 SSE指令简介

标量单精度浮点、组合单精度浮点指令

数据传送

算术运算

比较运算

逻辑运算

重排和解组

转换





2 SSE指令简介

SSE 64位SIMD整数指令

- 采用64位的 MMX寄存器和64位的内存操作数
- 保留了 MMX 指令
- 同时增加了一些新的指令



华中科技大学

2 SSE指令简介

MXCSR状态管理指令

缓存控制指令





3 SSE2及后续版本的指令简介

在SSE之后，出现了SSE2、SSE3、SSE4等版本
组合和标量双精度浮点指令
64位和128位整数指令





4 SSE編程示例

```
#include <stdio.h>
int main(int argc, char* argv[])
{
    float x, y, z;
    x = 3.14;
    y = 5.701;
    z = x + y;
    printf("%f\n", z);
    return 0;
}
```



4 SSE编程示例

x = 3.14;

movss xmm0,dword ptr [__real@4048f5c3 (0E67B34h)]

movss dword ptr [x],xmm0 标量单精度浮点数传送

..... **Scalar Single-precision floating-point**

z = x + y;

movss xmm0,dword ptr [x]

addss xmm0,dword ptr [y] 标量单精度浮点数加法

movss dword ptr [z],xmm0

printf("%f\n", z);

cvtss2sd xmm0,dword ptr [z] 数据类型转换

sub esp,8

movsd mmword ptr [esp],xmm0 标量双精度浮点数传送

push offset string "%f\n" (0E67B30h)

call _printf (0E61046h)

add esp,0Ch





4 SSE编程示例

.XMM ;处理器选择伪指令,支持SSE、SSE2、SSE3指令集

.model flat, stdcall

ExitProcess proto stdcall :dword

includelib kernel32.lib

printf proto c :ptr sbyte, :vararg

includelib libcmnt.lib

includelib legacy_stdio_definitions.lib

.data

lpFmt db "%f",0ah, 0dh, 0

x real4 3.14

y real4 5.701

z real4 0.0

.stack 200





4 SSE編程示例

```
.code
main proc c
    movss    xmm0, z
    addss    xmm0, y
    movss    z, xmm0
    cvtss2sd xmm0, z
    sub      esp, 8
    movsd    mmword ptr [esp], xmm0
    invoke   printf, offset lpFmt
    add      esp, 8
    invoke   ExitProcess, 0
main endp
end
```





4 SSE编程示例

若将 `z` 的定义改为 `z real8 0.0`,
程序中的片段可简化:

```
movss    xmm0, x
addss    xmm0, y
cvtss2sd xmm0,xmm0
movsd    z, xmm0
invoke   printf, offset lpFmt, z
```

`printf` 显示浮点数时, 要求一个双精度浮点数





5 用C语言编写SSE应用程序

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include <conio.h>
#include <emmintrin.h>
#define LEN 100000 // 数组大小
```





5 用C语言编写SSE应用程序

```
int main() {  
    clock_t stTime, edTime;  
    int i, j;  
    _declspec(align(16)) unsigned short a[LEN], b[LEN];  
    _declspec(align(16)) unsigned short c[LEN];  
    __m128i *pa, *pb, *pc;  
    int LEN8;  
    srand(time(NULL));  
    for (i = 0; i < LEN; i++) {  
        a[i] = rand(); b[i] = rand();  
    }  
}
```





5 用C语言编写SSE应用程序

```
    stTime = clock();
    for (j = 0; j < 1000; j++) {
        pa = (__m128i *)a;    pb = (__m128i *)b;
        pc = (__m128i *)c;
        LEN8 = LEN / 8;
        for (i = 0; i < LEN8; i++) {
            *pc = _mm_adds_epu16 (*pa, *pb);
            pa += 1;    pb += 1; pc += 1;
        }
    }
    edTime = clock();
    unsigned int spendtime = edTime - stTime;
    printf("time used: %d \n", spendtime);
    return 0;
}
```





5 用C语言编写SSE应用程序

如只有 SSE的指令，可以使用头文件`xmmintrin.h`。
在该文件中可以看到有SSE指令封装后的函数。

在VS2019平台中，有多个 `*intrin.h`，
它们对应着不同版本的指令封装。





华中科技大学

SSE 指令集

Streaming SIMD Extensions

数据位数更多的寄存器：128位的寄存器：xmm0～xmm7

更快的运算速度，更多地指令





華中科技大學

AVX 指令集





华中科技大学

AVX 指令集

- 1 AVX技术简介
- 2 AVX指令简介
- 3 AVX编程示例





1 AVX技术简介

高级向量扩展 AVX

Advanced Vector eXtensions

- 2011年, 在MMX、SSE序列之后出现的SIMD增强版。

Intel酷睿处理器i3、i5、i7系列中使用

- 2013年, Intel发布了 AVX2

- 2016年推出了AVX-512

酷睿i7、i9等CPU中都支持AVX-512





1 AVX技术简介

- 8个256位的寄存器 $\text{ymm0} \sim \text{ymm7}$
- YMM寄存器的低128位可看成是一个XMM寄存器
- 直接使用寄存器的名字，即采用寄存器寻址方式访问组合整数、组合浮点数和标量浮点数。
- 不能用于寄存器间接寻址、变址寻址和基址加变址寻址，即不能用于寻址内存中的操作数。





1 AVX技术简介

半精度浮点数

- 用16个二进制位来存储
- 最高位为符号位，之后是指数部分（5位）、有效数字部分（10位）。
- 半精度浮点数无法进行加、减、乘、除等运算，主要是用来节约存储空间的。





1 AVX技术简介

乘法与加法混合运算

- 采用乘法与加法混合运算 (Fused Multiply Add, FMA) 指令，不会对乘法的结果做舍入处理，只有在得到最后加法的结果后，才会做舍入操作。
- 这样使用FMA指令可以提高乘法累加（如向量内积）运算的性能和精度。





2 AVX指令简介

- 在指令编码模式中采用了一种新的前缀(VEX)。
- 大多数AVX指令采用了三目运算符的格式:
VOP DesOp, SrcOp1, SrcOp2
- SrcOp1和SrcOp2为源操作数地址
- DesOp为目的操作数地址。



2 AVX指令简介

AVX指令集大致可分为三类

- 用新的表示方法但功能等效于SSE指令的升级版指令；
- 新引入的指令；
- 功能扩展指令，包括半精度浮点数变换、乘法加法混合运算指令和新的通用寄存器指令。





3 AVX编程示例

```
. 686P
. XMM
.model flat, c
    ExitProcess proto stdcall :dword
    printf      proto :vararg
    includelib  libcmtd.lib
    includelib  legacy_stdio_definitions.lib
.data
    lpFmt      db      "%f", 0ah, 0dh, 0
    x  real4   3.14
    y  real4   5.701
    z  real4   0.0
.stack 200
```





3 AVX編程示例

```
.code
main proc
    vmovss    xmm0, x
    vaddss    xmm0, xmm0, dword ptr y
    vmovss    z, xmm0
    cvtss2sd  xmm0, z
    sub       esp, 8
    vmovsd    qword ptr [esp], xmm0
    invoke    printf, offset lpFmt
    add esp, 8
    invoke    Exitprocess, 0
main endp
end
```





3 AVX编程示例

与C语言编程中使用MMX、SSE技术一样，
在C语言编程中可以使用AVX函数，以提高大规模
数据运算的能力。

具体的函数可以参考头文件`immintrin.h`。

